Deep Learning

Exercise 2: Numpy & MatPlotLib

Room: **BIN-1-B.01**
Instructor: Manuel Günther
Email: guenther@ifi.uzh.ch
Office: AND 2.54

Friday, February 28, 2025, 10:15 – 12:00

Outline

Evaluation of Klicker Survey of Last Week

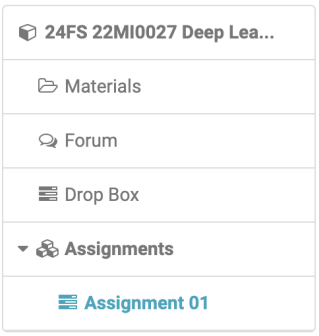
Aggregated Results

See Klicker Page online

(export currently not available)

Assignment Submission Details

Submission Portol



Submission

- Submit .ipynb file (no .pdf nor .py)
  - → Do not change the given template
  - → List group members on the top
- If work as team, submit by one person only
- Group members are not fixed (1-3 students)
- Submission deadline: Wednesday, 23:59

Modules

Modules in Python

- Library containing related functionality
- Can contain submodules, classes, functions
- Need to be `import`ed before usage

Preferred

```
import os
os.listdir(".")
```

Often Seen

```
from os import listdir
listdir(".")

import numpy as np
np.ndarray(5)
```

Highly Discouraged

```
from os import *
listdir(".")

from os import listdir as ls
ls(".")
```

Builtin Modules

Operating System `os`

- Directory handling
  → `getcwd`, `chdir`, `listdir`, `mkdir`, `rmdir`, `walk`
- File handling
  → `link`, `rename`, `remove`, `chown`, `chgrp`
- Filename handling `os.path`
  → `join`, `split`, `splitext`, `abspath`, `exists`, `isdir`
- Process handling
  → `fork`, `spawn`, `exec`, `nice`, `wait`, `kill`

Python System `sys`

- Command line arguments `argv`
- Search path `path`
- Console in/output
  → `stdin`, `stdout`, `stderr`

Mathematics `math`

- Constants: `pi`, `e`, `inf`, `nan`
- Rounding: `fabs`, `ceil`, `floor`
- Exponential: `exp`, `log`, `pow`, `sqrt`
- Trigonometrical: `sin`, `sinh`, `asin`

Builtin Modules

Random Numbers `random`

- Single value:
  → `random`, `uniform`, `gauss`, `randint`, `choice`
- Sequences
  → `shuffle`, `sample`
- Reproducibility: `seed`

Serealization `pickle`

- Writing: `dump`, `dumps`
- Loading: `load`, `loads`

Other Modules

- Temporary files: `tempfile`
- Compression: `tarfile`, `zipfile`
- Data Formats: `csv`, `json`, `html`, `pandas`
- Database Interface: `sqlite3`
- Argument parser: `argparse`, `getopt`
- Console/file logging: `logging`
- Parallelization: `multiprocessing`, `subprocess`, `threading`
- Communication: `urllib`, `http`, `ftplib`

Numpy Arrays

Basic Structure `array`

- Data container
  - → Contiguous memory storage
- `ndim`-dimensional in `shape/dims`
- One specific data type `dtype`
  - → default is `numpy.float64`
- Support of complex data
  - → `numpy.real`, `numpy.imag`
- Supports non-fixed types
  - → String, any `object`
  - → Reference (pointer) to objects

Data Types

- Integral Data
  - → `int8/16/32/64`
  - → `uint8/16/32/64`
  - → `bool`, `datetime`
- Floating point
  - → `float16/32/64/128`
- Complex data
  - → `complex64/128/256`
- Referenced types
  - → `str_`, `object`

Numpy Arrays

All functions / types in namespace `numpy`

Creating `array`s

- Creating in `dims`:
  - → empty: `empty(dims, dtype)`
  - → initialize: `zeros(dims, dtype)`
  - → range: `arange(min, max, step)`
- Creating from `data`
  - → `array(data, [dtype])`
  - → needs to be regularly structured
- **Avoid low-level function**
  - → `ndarray`

Random Data

- Uniform distribution:
  - → `random.random(dims)`
  - → `random.uniform(low, high, dims)`
- Normal distribution
  - → `random.randn(dims)`
  - → `random.normal(mu, sigma, dims)`
- Many more distributions
- `mu` and `sigma` can be vectors

Numpy Arrays

Indexing arrays

- Here: assuming 3D array `x`
  → with `dims=(3,6,4)`
- Single elements:
  → `x[0,0,0]`, `x[1,2,-2]` are OK
  → `x[3,0,0]` out of bounds
  → **Avoid** `x[0][0][0]`
- Multi-dimensional slicing
  → `x[1]`, `x[:2,:,2:]`
  → `x[1:3:2, 1::3, ::-1]`
  → Creates **views** of data

Array Methods

- Fill with value: `fill`
- Deep copy: `copy`
- Change shape: `reshape`
- Change to 1D: `flatten`
- Transpose: `transpose`

Array Manipulation

- `insert`, `append`, `delete`
  → Causes data copy, **avoid**
- Join matrices:
  → `stack`, `concatenate`

Math on Arrays

Element-wise Operations on Arrays

- Arithmetic: `+`, `-` , `*`, `/`, `//`, `%`
- Assignment: `+=`, `-=` , `*=`, `/=`, `//=`, `%=`
- Comparison: `<`, `>`, `<=`, `==`

  $\Rightarrow$ Boolean arrays

Broadcasting

- Repeating elements to higher dimensions
  - $\rightarrow$ Single value: `x += 1.`, `x > 0`
  - $\rightarrow$ Last dimension: `x * (1,4,3,2)`
- Dimensions must be equal or 1

Eamples

```
x = numpy.array([[1,2,3,4],
                 [5,6,7,8]])
# boolean arrays
x < 4
x == 4

# simple broadcasting
y = x * -1.
x *= -1.

# complex broadcasting
z = x * [[2],
         [-1]]
```

Math on Arrays

Arrays Reductions

- Reduce specific dimension (`axis`)
- `min`, `max`, `argmin`, `argmax`
- `sum`, `cumsum`, `mean`, `std`
- For Boolean arrays: `any`, `all`

Other Functions

- Sorting: `sort`, `argsort`
- Mathematical functions:
  - → `numpy.cos`, `numpy.tanh`
  - → `numpy.exp`, `numpy.log`
  - → `numpy.sign`, `numpy.power`

Eamples

```
x = numpy.array([[1,2,3,4],
                 [5,6,7,8]])
# minimum
x.min()
numpy.min(x, axis=1)

# sum and std
x.sum(axis=0)
numpy.std(x)

# boolean
(x > 8).any()

# sorting
numpy.argsort(x, axis=0)
```

Math on Arrays

Basic Linear Algebra

- Matrix multiplication: `numpy.dot`
- Vector multiplication:
  → `numpy.inner`, `numpy.outer`
- Norms: `scipy.linalg.norm`
- Inverse: `scipy.linalg.inv`
- Distances:
  → namespace `scipy.spatial.distance`
  → `euclidean`, `cosine`, `cdist`

Example

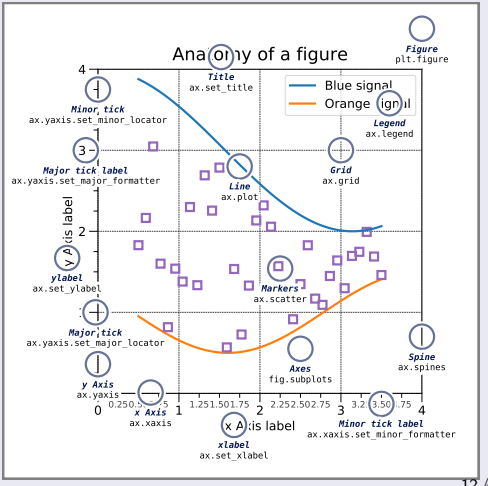
```
import numpy, scipy.spatial
# create two arrays
x = numpy.array([1.,2.,3.])
y = numpy.random.normal(size=3)

# compute vector prodcts
numpy.inner(x,y)
numpy.dot(x,y)
numpy.outer(x,y)

# normalize vector
x /= scipy.linalg.norm(x)

# compute cosine distance
scipy.spatial.distance.cosine(x,y)
```

Figures and Axes

Figures

- In namespace `matplotlib.pyplot`
- Basic visual element
- Incorporated into Jupyter
- Create figure (in inches)
  - → `figure(figisze=(width, height))`
- Can have one more `axes`
  - → by default one, get via `gca()`
- Can be saved to file
  - → `savefig(name, bbox_inches='tight')`
  - → Supports `.png` and `.pdf`

Anatomy of a Figure

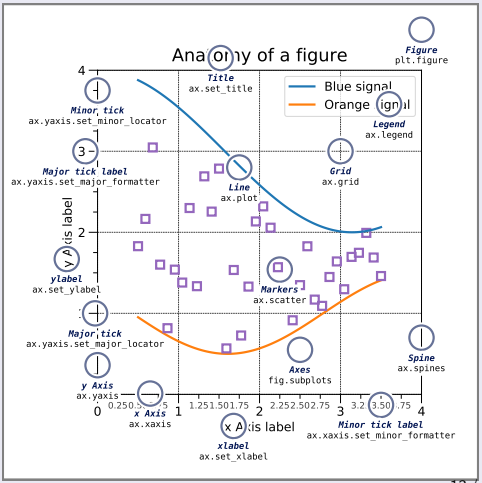


Figures and Axes

Axes

- Plotting area (by default 2D)
  - → how to plot: see next slide
- Axes labels: `pyplot.xlabel(text)`
- Major axes ticks and labels
  - → `pyplot.xticks(locations, text)`
- Axes limits `xlim(min, max)`
- Axis `title(text)`
- Axis `legend([labels], [loc])`
  - → Different labels, default: all lines
  - → Different locations, default: best fit

Anatomy of a Figure



Figures and Axes

Creating several `axes` inside `figure`

- Useful for comparing across plots, or showing images
- `fig, axs = pyplot.subplots([nrows], [ncols], [sharex], [sharey])`
  - → Creates and returns figure `fig` with axes `axs` in a `nrows * ncols` grid
  - → `axs` can be a single axis, a tuple of axes, or a tuple of tuple of axes
  - → Enable axes labels and ticks sharing across subplots
- `ax = pyplot.subplot(RCI, [projection], [sharex], [sharey])` or `ax = pyplot.subplot(R, C, I, ...)`
  - → Creates subplot with index `I` (one-based) of rows `R` and columns `C`
  - → `projection` enables different data projections, default: `rectilinear`

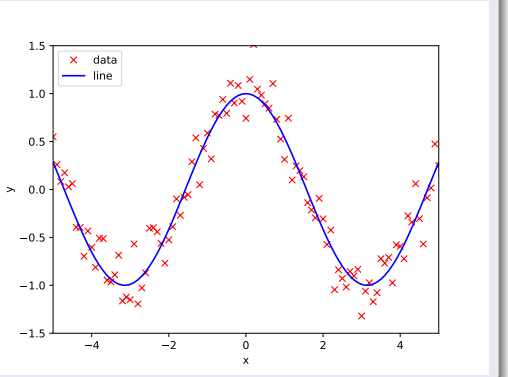
Plotting Functions

Plotting with `matplotlib`

- Basic interface: `plot([x], y, [format], [label])`
  - → Other plotting functions: `semilogx`, `boxplot`, `hist`, `pie`, `imshow`
- `x` and `y`: coordinates to plot; default: `x = range(len(y))`
- `format` mixes style and color in string
  - → Colors: one of `rgbkmcyw`
  - → Style: lines `-`, `--`, `-.`, `:` and markers `.,o+*xXspdv^<>`
- `label`: string for legend
- Lots of other parameters
  - → `alpha`, `color`, `drawstyle`, `linestyle`, `linewidth`, `dashes`
  - → `marker`, `markersize`, `markeredgewidth`, `markeredgecolor`, `markerfacecolor`

Plotting Functions

Example Plot



Example

```
import numpy
from matplotlib import pyplot

# create data
x = numpy.arange(-5,5.001,0.1)
target = numpy.cos(x)
data = target + numpy.random.normal(0, .2, x.shape)

# plot data points and line
pyplot.plot(x, data, "rx", label="data")
pyplot.plot(x, target, "b-", label="line")

# limit axes and provide labels
pyplot.xlim((-5,5)); pyplot.ylim((-1.5,1.5))
pyplot.xlabel("x"); pyplot.ylabel("y")

# create legend and write to file
pyplot.legend(loc="upper left")
pyplot.savefig("Example.pdf")
```

Plotting Functions

3D Axis in `matplotlib`

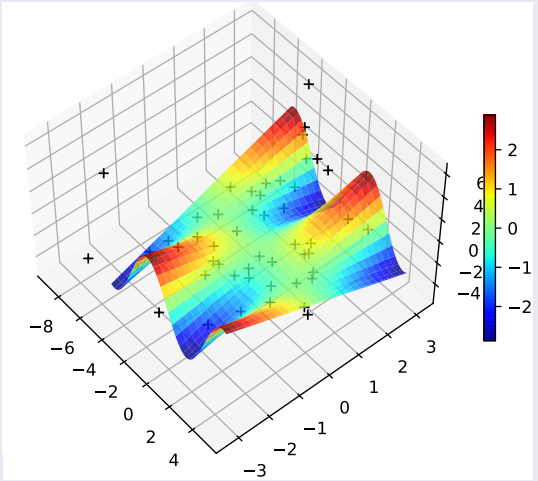
- Create a new figure `fig = figure([figsize])`
- Create a single 3D subplot Axis
  - → `ax = fig.add_subplot(111, projection="3d", [azim], [elev])`
  - → Creates a `mpl_toolkits.mplot3d.axes3d.Axes3D` object
  - → 3D camera location can be varied via `azim` and `elev`

Create Surface Plots

- Create 3D data: `xx, yy = numpy.meshgrid(..., ...)` and `zz = f(xx, yy)`
- Create surface plot: `ax.plot_surface(xx, yy, zz), cmap="..."`
- Add 3D points via `ax.plot(xs, ys, zs, ...)`

Plotting Functions

Example Surface Plot



Example

```
# create data
x = numpy.arange(-5., 5.001, 0.1)
y = numpy.arange(-3., 3.001, 0.1)
xx, yy = numpy.meshgrid(x,y)
zz = numpy.sin(xx) * yy

# surface plot
fig = pyplot.figure()
ax = fig.add_subplot(111, projection='3d',
            azim=-40, elev=50)
surface = ax.plot_surface(xx, yy, zz,
            cmap="jet", alpha=.8)

# create and plot 50 random 3D points
pts = numpy.random.normal(0, (3,1.5,3), (50,3))
ax.plot(pts[:,0], pts[:,1], pts[:,2], "kx")

# add color bar
pyplot.colorbar(surface, shrink=0.5)
```